

Chapter 8. Components of the Cloud Stack

Reasoning about Java performance on a single machine is difficult—there are many variables arising from the JVM subsystems and the underlying hardware. Prior to this chapter, we have explored and discussed how to approach these challenges. We've discussed some aspects of JVM internals, diagnostics, and operating system performance tools and how they help to interrogate a running process. Going further, mechanical sympathy—understanding the interaction between the JVM and hardware—allows us to address high-performance concerns on a single JVM.

In this chapter, we are going to break the single JVM model and look at platforms supporting a horizontal deployment model for Java processes. You will see how platforms hosting Java processes have significantly shifted. Specifically, cloud native environments have altered the landscape and, with that, the categorization of topics that architects and performance engineers need to understand.

In particular, in addition to the key questions highlighted in [“A Taxonomy for Performance”](#), there are situations where developers working in cloud-based platforms will also need to consider:

- Optimization for cost
- Optimization for reliability
- Scaling horizontally

In other words, optimizing for cost, reliability, and elastic scale (managing performance across multiple instances of running Java processes) will be key factors in complementing the classic taxonomy for performance.

In this chapter, you will learn an overview of some of the key cloud native building blocks and associated standards. You will also learn about Java standards relevant to building cloud native applications. We will cover a primer on virtualization, containers, and images.

We will then cover networking, as there are some major differences that influence the way that you will need to consider designing for cloud native. Finally, we will introduce the Fighting Animals repository, which we

will use in subsequent chapters to practically acquaint you with new concepts.

Java Standards for the Cloud Stack

Frameworks in Java extend the core Java libraries offered in the JDK to assist in solving real-world problems. This assists developers in solving common problems on common deployment targets and platforms.

Distributed platforms based on microservices-based architectures have become more prevalent. It is important to consider not just a single framework but also the available standards that apply to distributed deployment methodologies.

Standards create portability across a range of cloud native Java products including Quarkus, Helidon, and Open Liberty.¹

Of particular importance are these two open standards:

- [Jakarta EE](#), which we mentioned in [Chapter 6](#), provides a series of Enterprise Java standards and is widely adopted, but a full treatment of it is outside the scope of this book.
- [MicroProfile](#) is a standard for distributed systems on a cloud native platform, and as of version 6.1, effectively decomposes Jakarta EE 10 into a set of related, but independent, standardized components.

In particular, MicroProfile provides a set of vendor-neutral standards that support microservice based-architectures and distributed system best practice. It is of particular interest to Java developers and architects because it provides standardization for libraries for building [twelve-factor apps](#). Without standards, it is easy to end up in a situation where developers build their own solution or potentially get locked into a particular framework.

NOTE

The Eclipse Foundation is the home of both the MicroProfile and Jakarta EE working groups. These working groups are responsible for defining enterprise Java and microservices standards, respectively. Eclipse also hosts the Adoptium community build of OpenJDK.

[Figure 8-1](#) describes the standards covered in MicroProfile 6.1 and demonstrates the key aspects you must consider when building applications in cloud native environments. As trends evolve to different pat-

terns for building microservices-based architectures, Jakarta EE and MicroProfile will adapt and likely add new standards.

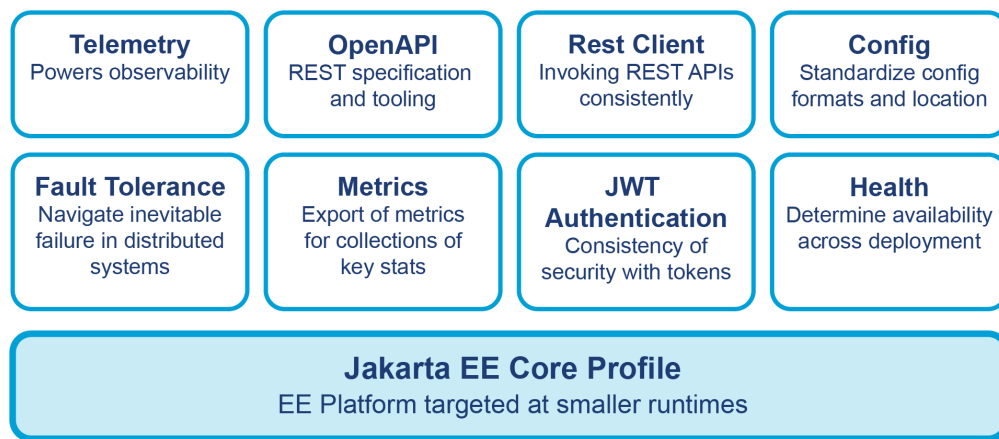


Figure 8-1. Structure of the MicroProfile standard

Standards in Java are useful, but we also need to find a strategy to address vendor neutrality and portability in the platforms we target. Open source software has a long tradition of using open foundations to address these aspects of the software landscape.

Cloud Native Computing Foundation

The Cloud Native Computing Foundation (CNCF) is a vendor-neutral open source software foundation dedicated to making cloud native computing universal.

Cloud native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.

—CNCF Charter

As vendor neutrality and portability is a significant architecture concern, it should not be a surprise that several CNCF projects are extremely important in the delivery of cloud native applications.

NOTE

Compute platforms are not tied to a specific language stack, so advice given in the remainder of this chapter will expand beyond the scope of Java.

When building systems composed of multiple services, it is likely that you will need to revisit where and how certain components are deployed to meet evolving nonfunctional requirements. This is where CNCF is critical, hosting critical projects with different benefits to help meet business and nonfunctional requirements.

Figuring out which of the many projects to apply to your specific use case is tough. The [CNCF Landscape](#) is an interactive map that attempts to categorize most of the projects and product offerings in the cloud native space. The CNCF Landscape is organized into multiple categories covering a range of concerns. Five of the key categories are:

- Application definition and deployment
- Orchestration and management
- Runtime
- Provisioning
- Observability and analysis

Within each of these categories in the CNCF Landscape, you will find the individual CNCF projects, the statistics of the project, and where the ownership of each project resides.

Note that the formats and standards for container images themselves are not part of the CNCF and are maintained by a separate standards initiative, the [Open Container Initiative \(OCI\)](#). Although vendor neutral, projects on the CNCF Landscape include “for profit” projects. CNCF open source projects have different levels of maturity, which assists in developers and architects considering various technologies in their cloud native deployments:

Sandbox

Early innovative projects are at an early stage of development and may not yet be at production standard.

Incubating

The project is ready for production and has been demonstrated by active adoption. The project has adequate governance and maintenance, community, engineering principle, security, and ecosystem. It should meet the criteria of the [incubating template](#).

Graduated

The project has a proven track record of production usage in multiple industries and projects. It should meet the criteria of the [graduation template](#).

Architects frequently use the CNCF landscape. It is used as a mechanism to identify technologies in a specific problem area, discover the merits of the project, and for focusing proof of concepts and spikes of functionality.

Three CNCF projects are critically important to cloud native Java developers and performance engineers.

Kubernetes

Kubernetes (often shortened to K8s) is an open source container-orchestration system. It uses a cluster of compute nodes (hosts) to enable system operators and DevOps teams to deploy, scale, and coordinate distributed applications across the cluster. Kubernetes became a graduated project in CNCF in 2018.

In [Chapter 9](#) you will learn about deploying Java applications using containers and Kubernetes.

Prometheus

Prometheus is a metrics format and time series database that is used to store metrics data. It was accepted to the CNCF in May 2016 as an incubating project and achieved graduated status in August 2018.

It is widely deployed among Kubernetes applications and has benefited from significant first-mover advantage, although the metrics landscape is evolving rapidly. You will learn more about Prometheus in [Chapters 10](#) and [11](#).

OpenTelemetry

OpenTelemetry (OTel) is a set of standards, formats, and libraries that handles the collection, aggregation, and transport of observability data from applications into an observability system.

[OTel](#) is a CNCF project, and the technical development of the project takes place on [GitHub](#).

OTel is explicitly a cross-platform technology and is not Java-specific, although Java is a mature implementation of the standards. This means that there are a lot of different projects written in (and for) multiple programming languages.

OTel is currently an incubating project at CNCF, but it is seeing explosive growth and is already being used in production by many organizations. We will revisit OTel in depth in [Chapter 11](#).

Underpinning the aforementioned technologies is the model for how things are deployed, so let's look at the importance of virtualization in the cloud native stack.

Virtualization

Before we can address the topic of virtualization, we first need to address the broader question—what is *cloud*? You will often hear the quip, “*Cloud is just someone else's computer*,” but there is more to it than that. The following helps provide a working definition:

The simplest definition of cloud is a data centre that's full of identical hardware that no-one ever touches except to unpack it on day one and throw it away when it fails; in between, every deployment, update, investigation, and management process is automated.

—[Mary Branscombe](#)

When your infrastructure is in the cloud, your capacity is commoditized and ready to run, available across a series of diverse application use cases. Access is not typically provided directly to the infrastructure and hardware. Instead, there needs to be control, management, and clear separation of the customer's runtime from both the infrastructure and potentially other customers.

For Java developers and performance engineers, this is our first tradeoff related to the move to cloud.

Access to the underlying operating system and hardware is available only at significant cost; typically you will have only limited insight into the physical platform, if at all. Perhaps surprisingly, virtualization techniques were originally developed in IBM mainframe environments as early as the 1970s. However, it was not until recently that x86 architectures were capable of supporting “true” virtualization.

TIP

The traditional sysadmin techniques of “SSH into a box and have a look around” are not normally available in cloud environments—much more, arm's-length management techniques have become the standard approach.

Virtualization is typically defined by the following three conditions:

- Programs running on a virtualized OS should behave essentially the same as when running on bare metal.²
- A component, known as a *hypervisor*, must mediate all access to hardware resources.
- The overhead of the virtualization must be as small as possible and not a significant fraction of execution time.

In a traditional, unvirtualized system, the OS kernel runs in a special, privileged mode (hence the need to switch into kernel mode). This gives the OS direct access to hardware—this is the situation when working locally on your developer laptop, for example.

However, in a virtualized system, direct access to hardware by a guest OS is prohibited. [Figure 8-2](#) outlines the structure, with the host operating system of the commoditized cloud infrastructure forming the base layer of the infrastructure. The `Feline` and `Mustelid` components are typical REST microservices that we will introduce later in the chapter.

The next layer is the hypervisor, which acts a layer of indirection between the operating system and the guest operating system. As a developer, you can deploy freely to the guest operating system, potentially using containers—we will discuss this option shortly.

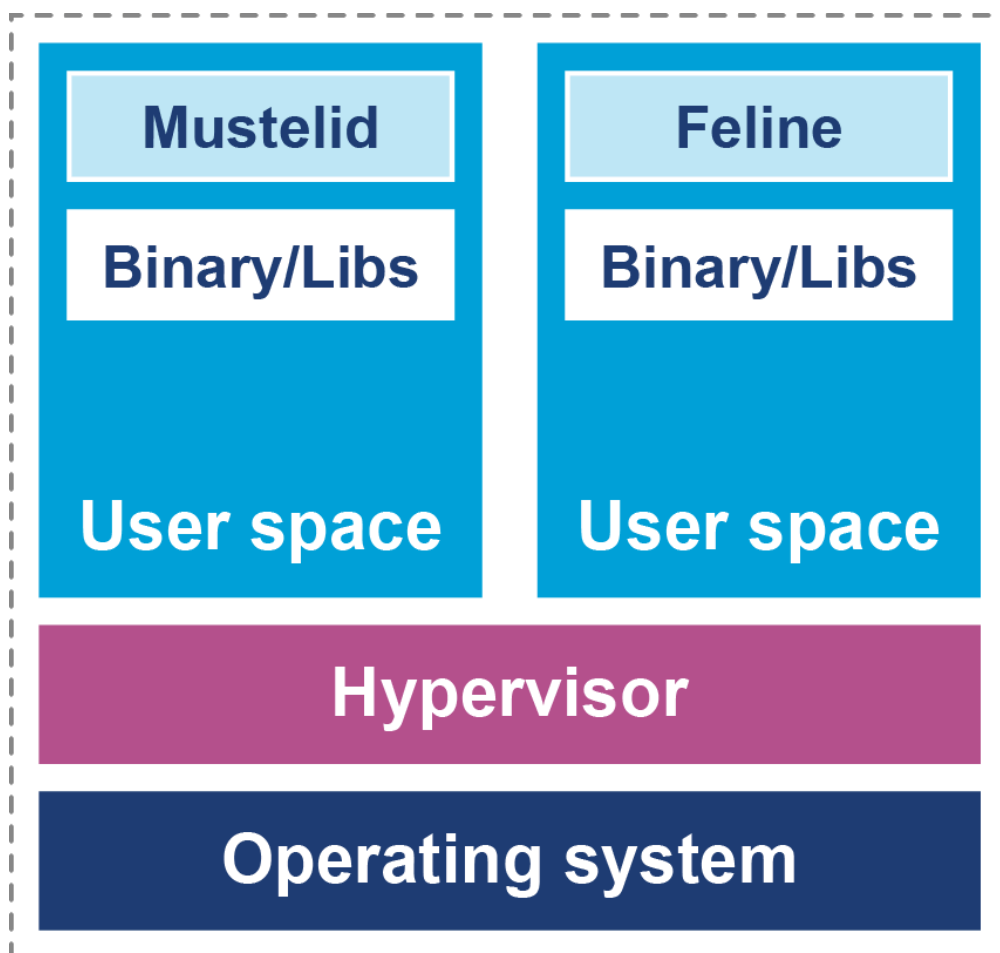


Figure 8-2. Virtualization stack

The hypervisor is at the core of virtualization, and it could be easy to look at it and remember tales of slow virtualized environments. The extensive use of virtual machines and hypervisors in the cloud has driven significant research and improvement to the overhead of hypervisors.

When moving to public cloud, your target platform will typically be a virtual machine packaged with the hypervisor. This will have an overall impact on performance that is, at least partly, out of your control—although public cloud providers do offer different options for the virtual machines available to you. Choosing the right virtual machines will become a factor for certain types of workload that run on public cloud.

Let's explore some of the virtual machines offered by cloud providers and what decisions you have as an architect and developer.

Choosing the Right Virtual Machines

Public cloud providers offer different types of virtual machines that are designed to suit different types of workload profile.

[Amazon EC2](#) has a huge number of possible options for VM configuration depending on your use case. AWS also offers bare metal servers in addition to virtual machines; the following are some EC2 options available for running workloads:

General purpose

The starting point for applications; however, within general purpose, there is a wide range of compute options. For example, in the M7g series, you can choose from a medium VM with a single vCPU and 4 GB of memory to the `m7gd.metal`, a bare metal machine with 64 CPU and 256 GB of memory.

Compute optimized

Offer support for workloads with high CPU load.

Memory optimized

Setup for workloads with large data sets with significant in-memory read and write profiles. This category includes databases, in-memory caching, and data analytics.

Accelerated computing

Uses hardware acceleration and is geared to graphics, intense calculations, and generative AI applications.

Storage optimized

Designed for read and write operations on local storage. This type of instance is focused on transactional databases and Apache Spark-style workloads.

High-performance computing

Workloads are focused on complex simulations and deep learning-style workloads.

[Azure](#) provides a wide range of options, from D-series used for general-purpose compute to L-series used for storage-optimized virtual machines.

[Google Cloud](#) has a similar set, including workloads focused on general purpose, ultra-high memory, compute-intensive, and demanding application workloads that require GPUs.

As part of designing your applications on cloud, generally the best approach is to start with general-purpose VMs and build from there. Instance types are often sized in powers of 2, so moving to bigger VMs is a cost tradeoff in situations where you just need a 5% extra CPU. Instance families provide a consistent ratio of CPU/memory/network throughput/storage (if they have it) across the instance sizes in that family. This can be a helpful starting point for picking an instance type.

Remember the lessons from earlier in the book—take small steps and measure whether adjusting VMs provides the type of performance benefit versus cost tradeoff that you are looking for.

In [Chapter 9](#), you will explore how to further mix and match VMs within your application deployments, as part of the challenge will boil down to scheduling and deploying cloud-based applications. Orchestration platforms like Kubernetes provide the ability to mix and match VMs, so you can potentially run one specialized pool of nodes while other nodes focus on general-purpose use cases.

Virtualization Considerations

When designing cloud applications, optimize for cost, reliability, and scalability by choosing the right VM for your use case. For example, in situations where you are (or expect to be) CPU-intensive, this impact should be measured and confirmed as part of a performance test conducted in a production-like environment.

One additional benefit of running in cloud is that creating ephemeral environments for testing is simpler. We will revisit this concept in [Chapter 9](#).

Another option is to run a “best of both worlds” platform. In this model, some processes run on public cloud and some processes on bare metal. This provides the possibility of optimizing certain workloads that need burst scale and reliability, but also allows for performance and consideration of mechanical sympathy on core processing.

We will explore this further in [“Container Orchestration”](#), when we look at orchestration systems and placement of processes, but Red Hat’s OpenShift (“hybrid cloud”) technology is a good example of this approach.

Now that you have an understanding of VMs, let’s take a closer look at the building blocks for cloud native deployments, images, and containers.

Images and Containers

When Java arrived in the late 90s, it promised the hope of a great future of portability with its mantra of *“Write once, run everywhere,”* meaning any operating system and machine capable of running a Java virtual machine can run your code. This was an extremely audacious ambition, and the abstraction was not always perfect, especially in the early days.

Nevertheless, as with so many other aspects of modern software, Java served as the conduit by which advanced ideas truly entered the mainstream.

Big ideas such as virtual machines, dynamic self-management, JIT compilation and garbage collection are now part of the general landscape of programming languages.

—Benjamin J. Evans, “Java is a ’90s Kid” in *97 Things Every Java Programmer Should Know*

The technology landscape continues to shift, and there has been an explosion in programming languages and platforms targeted at big data, artificial intelligence, cloud native routing, and network-level products. To support this array of complex technologies, the industry has had to react to deploying software on various operating systems, environments, and with a more diverse set of dependencies.

Portability is a design goal of cloud native applications, although it has appeared not in the portability of Java bytecode but at a slightly higher level—the *container image*. A container image (or just *image*) is an archive file that can be used to create an application process (or group of them) running under the control of an orchestration or container management system.

Let's dive deeper into image structure.

Image Structure

Just as in traditional Unix environments, the executable file on disk is the “frozen” representation of the program. At startup, this is converted into an active application process by executing the program. The image can be thought of as the frozen (and portable) representation of the application component, which will be converted into an active component via scheduling and orchestration.

With this greater diversity and complexity of components, standardization is once again the weapon of choice that industry uses to manage and control their application stacks. A good example is Open Container Image (OCI) established in 2015 by Docker.

Images are becoming the industry preferred unit of packaging applications, including Java applications, even if the target platform is bare metal. The image is bundled with everything required to run the application, which includes the userspace components (such as a subset of operating system components) and the JVM.

Accordingly, OCI is responsible for defining the format of images, how images run as containers, and how images are distributed.

Let's explore some of the interesting aspects of building and running containers—and some of the potential issues to be aware of.

Building Images

One way to create an image as part of the software build process is to define the image instructions in a `Dockerfile`. Each line in the `Dockerfile` represents a new layer. A layer is an immutable change to the file system, which will be represented within the container and is stored in the build cache. Each existing layer in the build cache can be reused as a building block for new images.

NOTE

Docker is often confused between the use as a technology and the commercial entity and registry provider. In this chapter, we refer to Docker as the de facto standard format that is supported in open source and by excellent tooling.

In the following example, we can see the keyword `FROM`, which essentially uses another image layer as a basis for adding in our Java application `animals-demo-1.0-SNAPSHOT.jar`.

The `USER`, `RUN`, `COPY`, and `WORKDIR` commands set up the app folder and move the built `jar` to the `/app` folder, ready to be executed by the container entry point command `CMD`:

```
FROM registry.access.redhat.com/ubi8/openjdk-17:1.13-1
USER root
RUN mkdir /app
COPY target/animals-demo-1.0-SNAPSHOT.jar /app
WORKDIR /app
CMD [ "java", "-jar" , "animals-demo-1.0-SNAPSHOT.jar", \
      "io.opentelemetry.examples.animal.AnimalApplication" ]
```

Other tools have been developed as alternatives to the Dockerfile approach, which have more information available to make build and layer optimization decisions.

[Jib](#) runs as part of a Java build system (e.g., Maven) to create the image and, therefore, has the advantage of having more information about the structure of your Java application and the dependencies. It organizes the image into distinct layers, including dependencies, resources, and classes. Jib only modifies the layers that have changed. The theory is that Java library dependencies and resources in the project change infrequently compared to the classes in your application code. By splitting layers out in this way, the immutable layers do not experience as much churn, decreasing the time of builds and startup during development.

Using Jib has the added benefit of keeping all dependencies fresh with each build of your application, for both Java-level libraries and lower image layers. Using the Dockerfile approach works too, but it is necessary to ensure that base images are updated as patches and newer versions are released. Jib has potential benefits for security patching and performance by assisting in keeping dependencies fresh.

It is also possible to run multistage builds in Docker by using it to both build the `jar` and then use the `jar` in a second stage to construct the

image. The layers used as part of the first build stage are discarded, and only the target `jar` is copied across into the final image. The advantage of this is ensuring a standardized build environment by containerizing and simplifying the build process. The following example Dockerfile demonstrates this multistage approach.

The first part of the Dockerfile sets up base images with Maven and OpenJDK 17 to execute the build, creating a `jar`. The `AS` builder identifies the stage in subsequent stages of the build. The second part of the Dockerfile builds the target image and includes the `jar` using the `--from=builder` statement.

```
#This first stage acts as a builder using a maven base image to create th
FROM maven:3.9-openjdk-17 AS builder
COPY src /usr/src/app/src
COPY pom.xml /usr/src/app
RUN mvn -f /usr/src/app/pom.xml package

FROM registry.access.redhat.com/ubi8/openjdk-17:1.13-1
USER root
RUN mkdir /app
#Copy the jar created by the first stage
COPY --from=builder /usr/src/app/target/animals-demo-1.0-SNAPSHOT.jar /ap
WORKDIR /app
CMD ["java", "-jar" , "animals-demo-1.0-SNAPSHOT.jar"]
```

A typical image stack for a Java application includes OS dependencies, the JVM and configuration, and a `jar`. A minimal base image is the [Universal Base Image \(UBI\) Minimal](#) from Red Hat, which is only 37.1 MB in compressed size. Layering OpenJDK on the AMD64 architecture results in a base image of 147.8 MB in compressed size. The final layer depends on the build size of your application. The images can get quite sizable; for example, a Windows AMD64 Eclipse Temurin image is 2.27 GB.

In [“Challenges with Containers and Scheduling”](#), we will consider how the size of an image has the potential to impact scheduling.

Running Containers

Containers provide a mechanism for running applications in an isolated environment. They are controlled using two Linux kernel constructs: *namespaces* and *cgroups*. Namespaces control access and visibility to resources on the host machine, and *cgroups* enforce limits to machine resources—especially CPU utilization and memory.

One aim of container abstraction is to provide process isolation between different containers. This is somewhat similar to virtual machines—the major conceptual difference is that containers do not use a hypervisor like VMs. Instead, applications in containers execute directly on the host operating system and access the host’s kernel without the additional indirection of a hypervisor.

This makes containers lightweight and quick to start on compute, forming the basis of the orchestration systems we will discuss in [Chapter 9](#).

Let’s now look at the importance of networking when running containers.

Networking

Containers and orchestration systems use *ephemeral compute*, so developers should expect workloads to not be reliably in the same “place” (i.e., physical or virtual host). Ephemeral compute has an advantage optimizing for cost, as it enables dynamic scale up and scale down.

Scaling up/down can be within a given footprint, or perhaps, dynamically adding to the footprint as needed (e.g., adding more nodes into a cluster). This can take advantage of spot computing, the ability to use unutilized plant resources at a reduced cost. Spot instances are not reserved or guaranteed; when demand for those resources occurs, the spot instances will be terminated without advance notice.

From a networking perspective, things won’t be as stable as fixed on-premises data centers, and components won’t always have a dedicated IP address or continuously available compute. As a result, when thinking about networking and application communication, it is often best to think about the abstraction of traffic. Traffic can be generalized into categories, north → south and east → west.

North → south is traffic that is not part of your system and will originate from either somewhere else on cloud or perhaps from the internet. North → south traffic needs to have a fixed IP address and is often referred to as an ingress. We will cover this in more detail in [“Connecting to Services on the Cluster”](#).

One option for ingress is to use a highly available load balancer, and many orchestration platforms utilize a load balancer supplied by the cloud provider. The individual applications would then set up the corresponding routing into the cluster. This allows for a fixed IP address for

external calls but still provides the ability to scale up and add more workloads behind the scenes to respond to scale.

East → west often refers to service-to-service calls within the orchestration tier, and here you can use the internal service discovery provided by the orchestration platform to scale up and down. Within an orchestration system like Kubernetes, we can use a service to create a lightweight and local DNS entry, available to other services within the cluster. We will revisit this in [“Services”](#) when we dive deeper into this topic.

To finish this chapter, let’s introduce a fleshed-out example system as a way to illustrate the concepts of cloud native applications, and later, build and deployment, and then observability.

Introducing the Fighting Animals Example

In this book, we have tried to ensure that we provide complete and working examples that go beyond “Hello World” but are still simple enough to be understood and used as a starting point for real systems. This is for several reasons, but one of the most important is that newcomers to the cloud stack frequently find it difficult to progress beyond initial, often semi-trivial, sample projects.

When observability is layered onto the example, the situation gets more complex, as overly simple examples do not always generalize easily to an actual implementation that can form the basis of an actual production observability system.

There is also a certain amount of irreducible complexity involved in implementing an observability system, and this can be tricky to set up, as there are so many variables and options that may or may not apply to a specific observed system.

Therefore, we have made the decision to introduce the “Fighting Animals” example application here first. After this, in Chapters [10](#) and [11](#), you will discover the observability considerations in more detail.

Fighting Animals is a simple Java application with a microservice architecture, [available on GitHub](#). The main version is written as a Spring Boot application, but there is an [alternative version](#) based on Quarkus. In the book text we will stick to the Spring Boot version, though.

The application is run as a collection of Docker containers (e.g., via Docker Compose) and exposes a REST endpoint on port 8080. Hitting the endpoint returns a simple JSON representation of two animals from several different biological clades that will battle each other.³

Call `GET /battle` to get a battle that looks like this:

```
{
  "good": <animal1>,
  "evil": <animal1>
}
```

There are several branches that we will explore further (mostly in the upcoming chapters):

main

No observability

micrometer_only

Micrometer metrics only

micrometer_with_prom

Micrometer metrics with Prometheus

manual_tracing

OpenTelemetry tracing using manual spans

auto_otel

Use of the OpenTelemetry Java agent to trace automatically

k8s-with-argo

An example using Kubernetes and rolling out change using deployment strategies

logging_only

SLF4J logging exported to OpenTelemetry

micrometer_with_otel

Micrometer with OpenTelemetry metrics

otel_metrics_raw_api

OpenTelemetry metrics using the raw API

Enhancing Fighting Animals with Kafka

Enhancing Fighting Animals with Kafka and Infinispan

In [Figure 8-3](#), we can see the simple structure and API invocations of the Fighting Animals system.

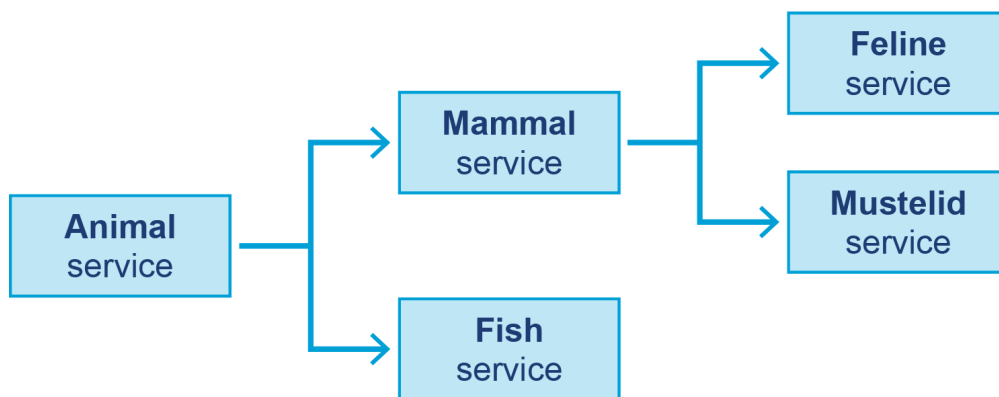


Figure 8-3. Fighting Animals microservices

The initial call results in further downstream calls to other microservices to fulfill the incoming request.

A WORD ABOUT VERSION NUMBERS

In all the Fighting Animals examples, we are using *pinned versions* of our dependencies and containers. This is to ensure the examples are reproducible and do not change over time. In a system that uses *floating versions* (like `latest`), the versions will change over time. This can make it very difficult to understand what has changed and why—and this problem only gets worse as the system gets more complex. We do use `latest` for our own application containers, but that's because they are under our control and we can ensure that they are always up to date.

Summary

In this chapter, you have learned about how cloud platforms and components challenge our traditional model of optimization and performance.

MicroProfile is an excellent standard for building applications that are both cloud native and microservice-based architecture ready. Beyond considering Java standards in isolation, CNCF provides a guide for plat-

form considerations and projects that should be considered for a cloud native architectural approach.

Virtualization is an important aspect of the cloud native stack and the principal building block for compute. Although there are low-level details in virtualization, the bigger benefits are gained from looking at the right VM configuration for the task. It is possible to mix bare metal and virtualization in a clustered deployment.

Images and containers are the de facto unit of deployment in a cloud native environment. There are different approaches to creating and layering images for Java and some common gotchas that need to be considered. Networking is an important aspect of the cloud native stack, and with that, the concerns of service discovery and traffic routing. We introduced the Fighting Animals project, which we will use throughout the upcoming chapters to describe more complexities in the cloud stack.

In the next chapter, we will look at running cloud native Java processes in more detail. We will start by looking at running containers locally and then how to schedule and deploy on cloud native infrastructure.

- 1** Some popular frameworks, such as Spring, have not historically participated in standardization efforts.
- 2** We will use “bare metal” to refer to running software directly on a host with a single operating system and no virtualization.
- 3** The application started life as a “battle” application that used characters from a well-known entertainment giant. This had to be changed for the book for fairly obvious copyright reasons.